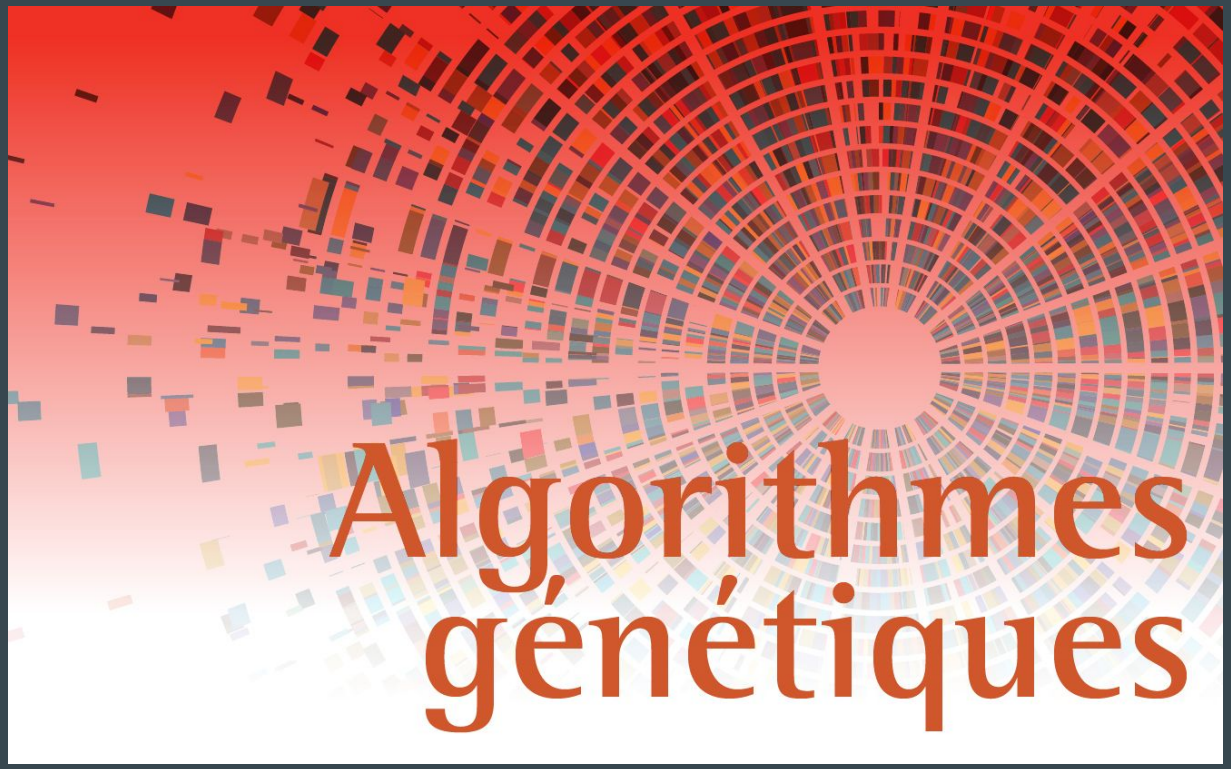
Algorithmes génétiques

L'objectif :

- Comprendre un algorithme génétique

- Programmation d'un algorithme génétique avec machine learning par moi même à travers le jeu rétro snake



Algorithmes génétiques

Plan :

- I Définition
- II Explication/Date
- III Application au projet

I Définition:

Les algorithmes génétiques :

- Appartiennent à la famille des algorithmes évolutionnistes

- Obtenir une solution approchée à un problème d'optimisation, lorsqu'il n'existe pas de méthode exacte pour le résoudre en un temps raisonnable



Solution of a travelling salesperson problem: the black line shows the shortest possible loop that connects every red dot.

II Explication/Date:

Découvert par :

- John Holland, ses collègues et élèves de l'Université du Michigan

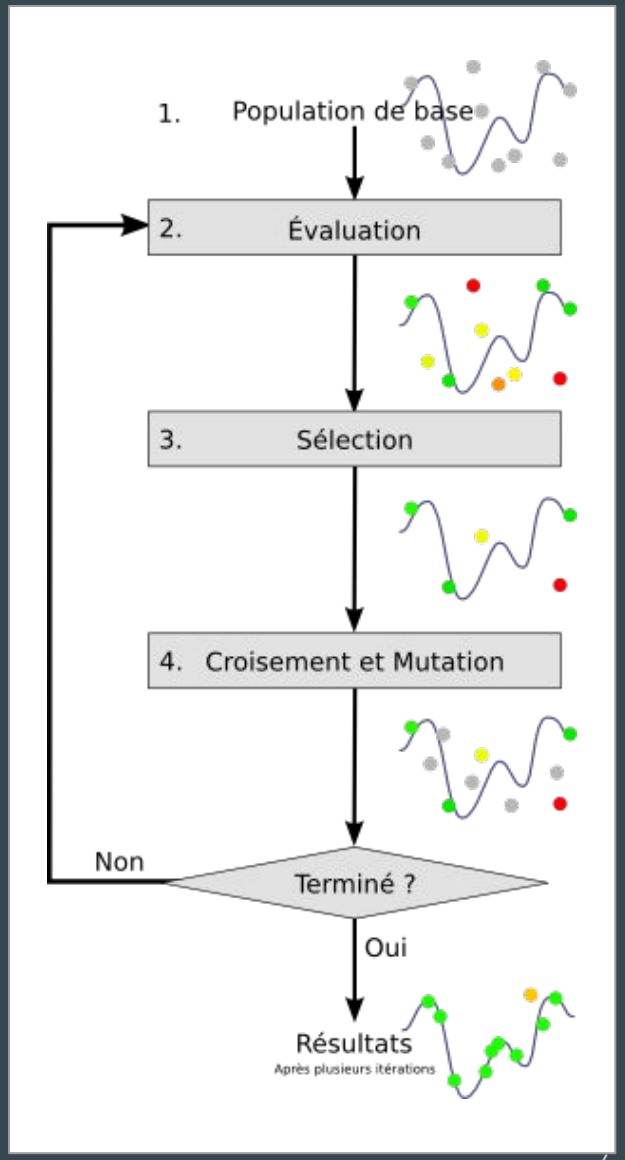
En :

- 1960

Ou :

- États Unis état du Michigan

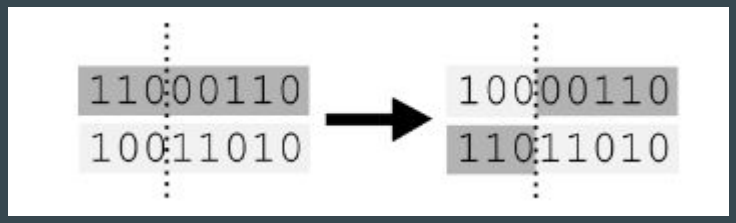


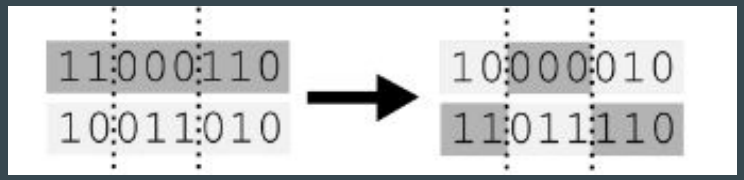


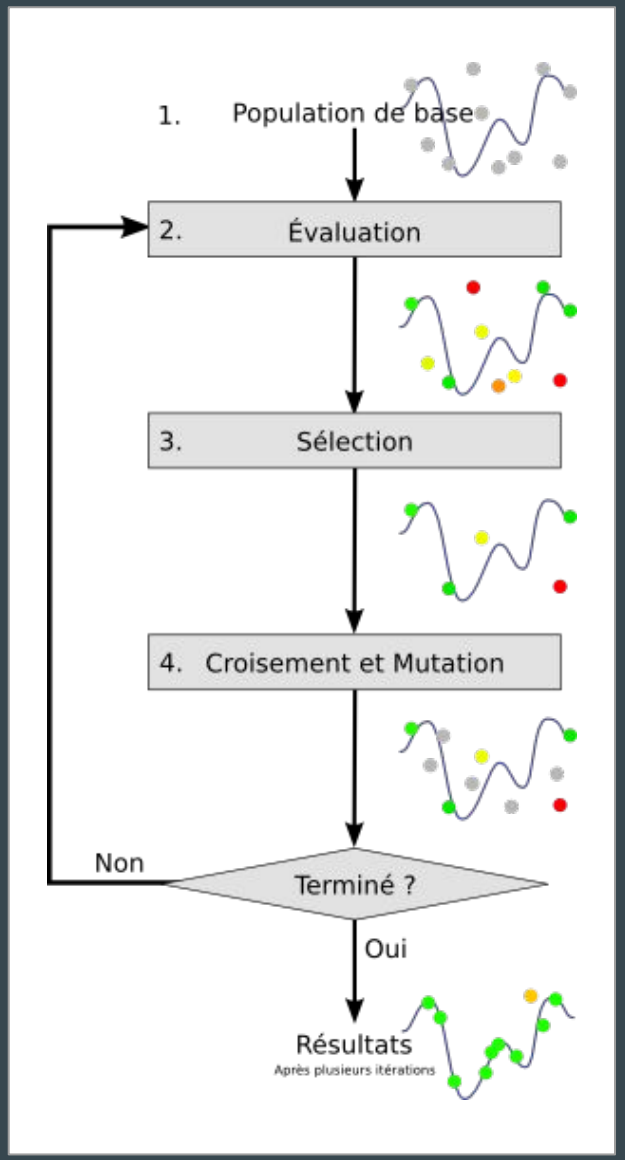
II Explication/Date:

Deux parents sont combinés pour créer un ou plusieurs enfants.

- One-Point Crossover : Un point de coupure est choisi aléatoirement, et les parties des parents sont échangées à ce point.
- Two-Point Crossover : Deux points de coupure sont choisis, et les segments entre ces points sont échangés.
- *Uniform Crossover : Chaque gène de l'enfant est choisi aléatoirement parmi les gènes correspondants des parents.



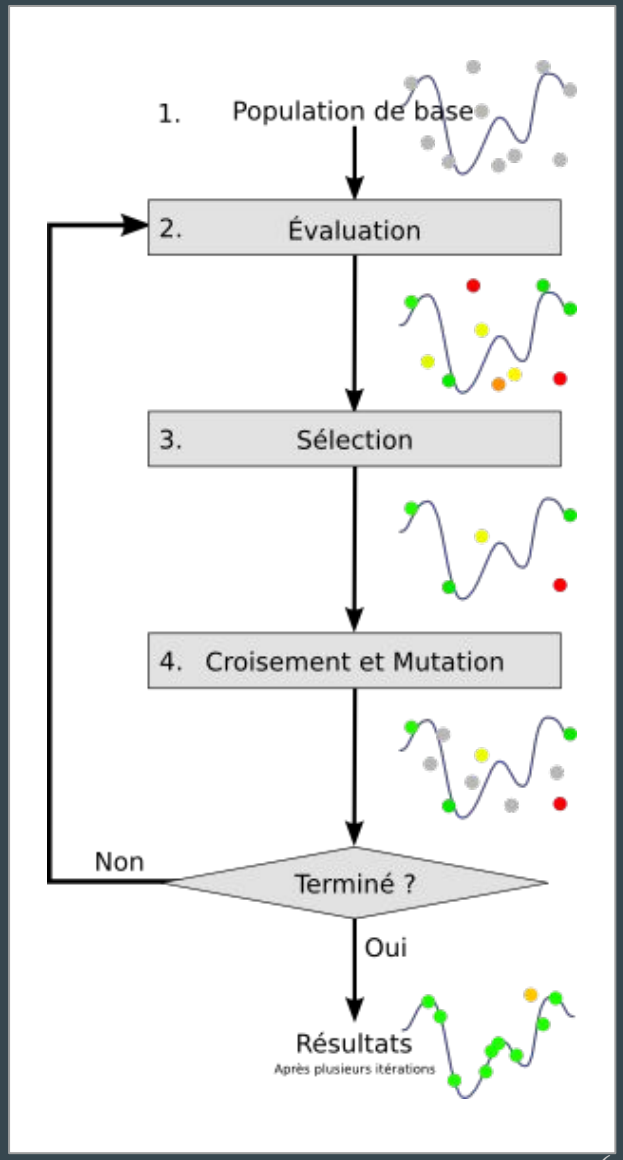




II Explication/Date:

Diversité génétique :

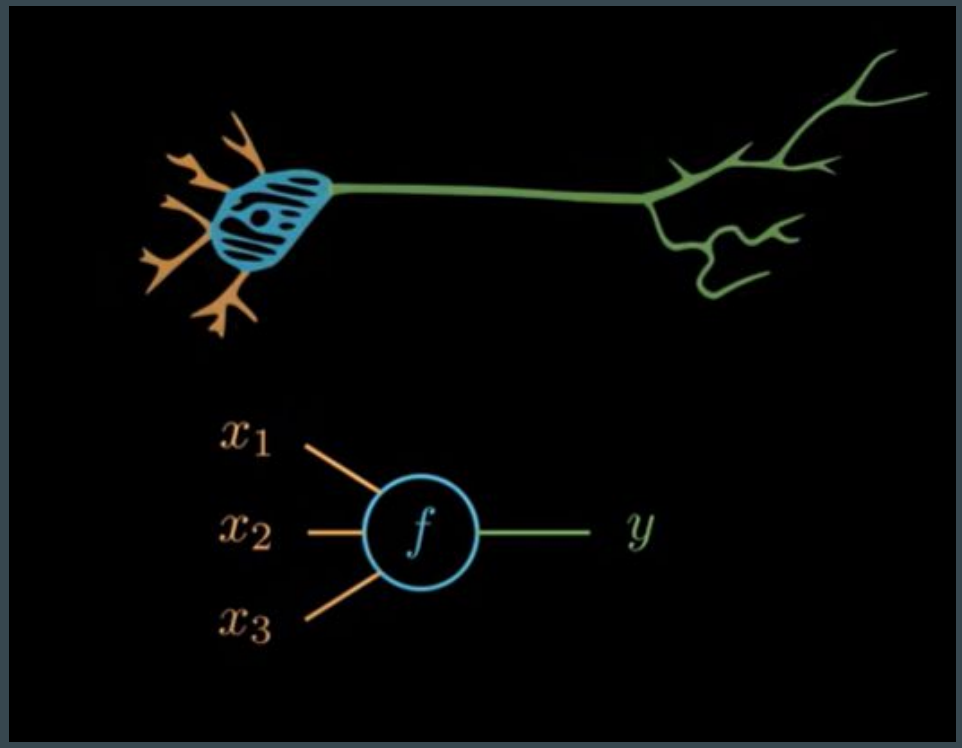
- *Mutation de Point : Un ou plusieurs gènes sont modifiés aléatoirement.
- Mutation de Swap : Deux gènes sont échangés.
- Mutation de Gaussienne : Pour des valeurs continues, des perturbations basées sur une distribution gaussienne sont appliquées.



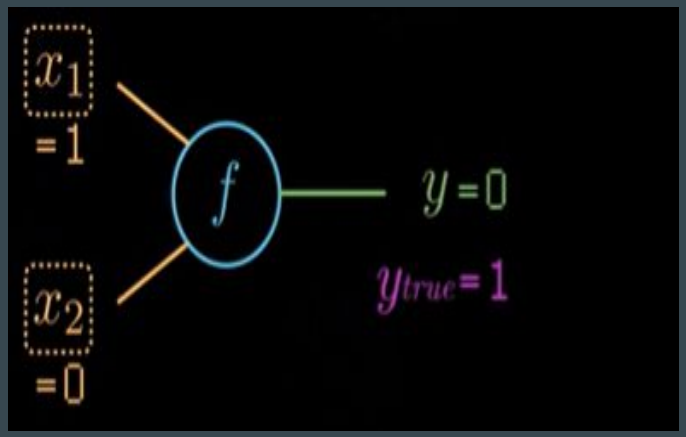
II Explication/Date:

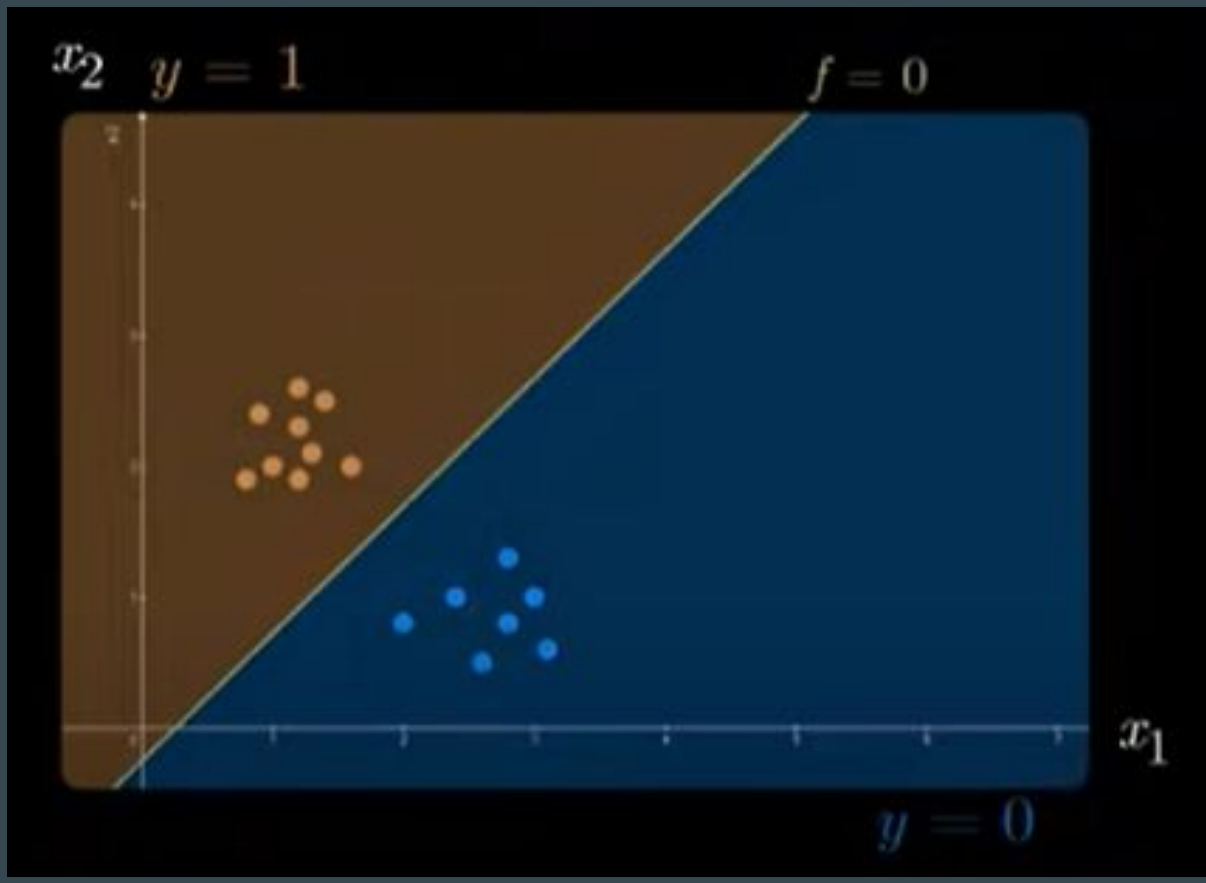
$$f = w_1x_1 + w_2x_2 + w_3x_3$$

w = coefficient excitateur varie en 0 et 1



II Explication/Date





$$W = W + \alpha(y_{\text{true}} - y)X$$

W = le poids

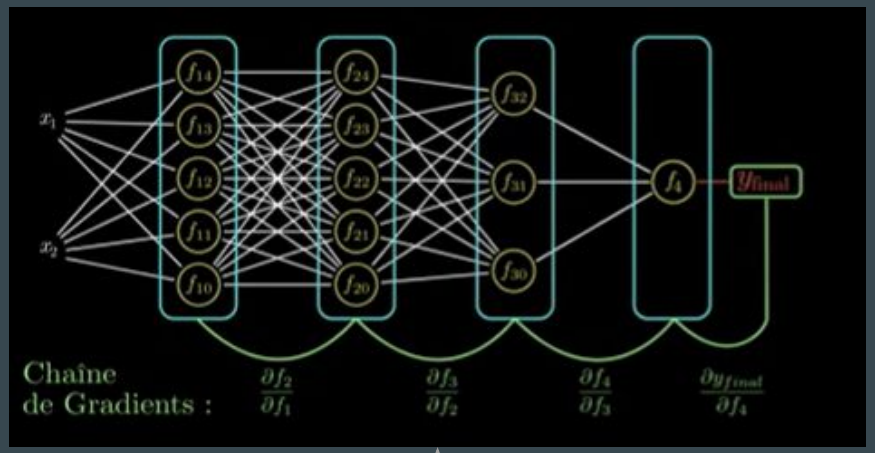
α = learning rate

y_{true} = est la vrai valeur

y = la valeur prédite par le modèle d'entrée

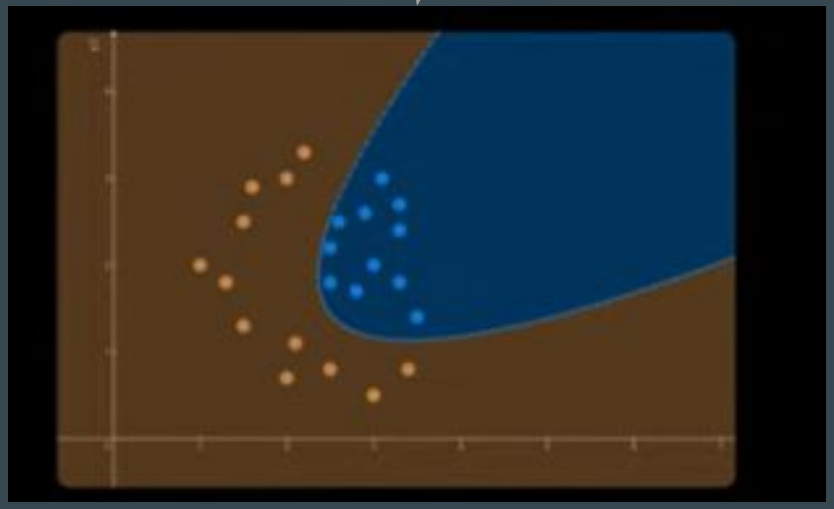
X = les données d'entrées

II Explication/Date



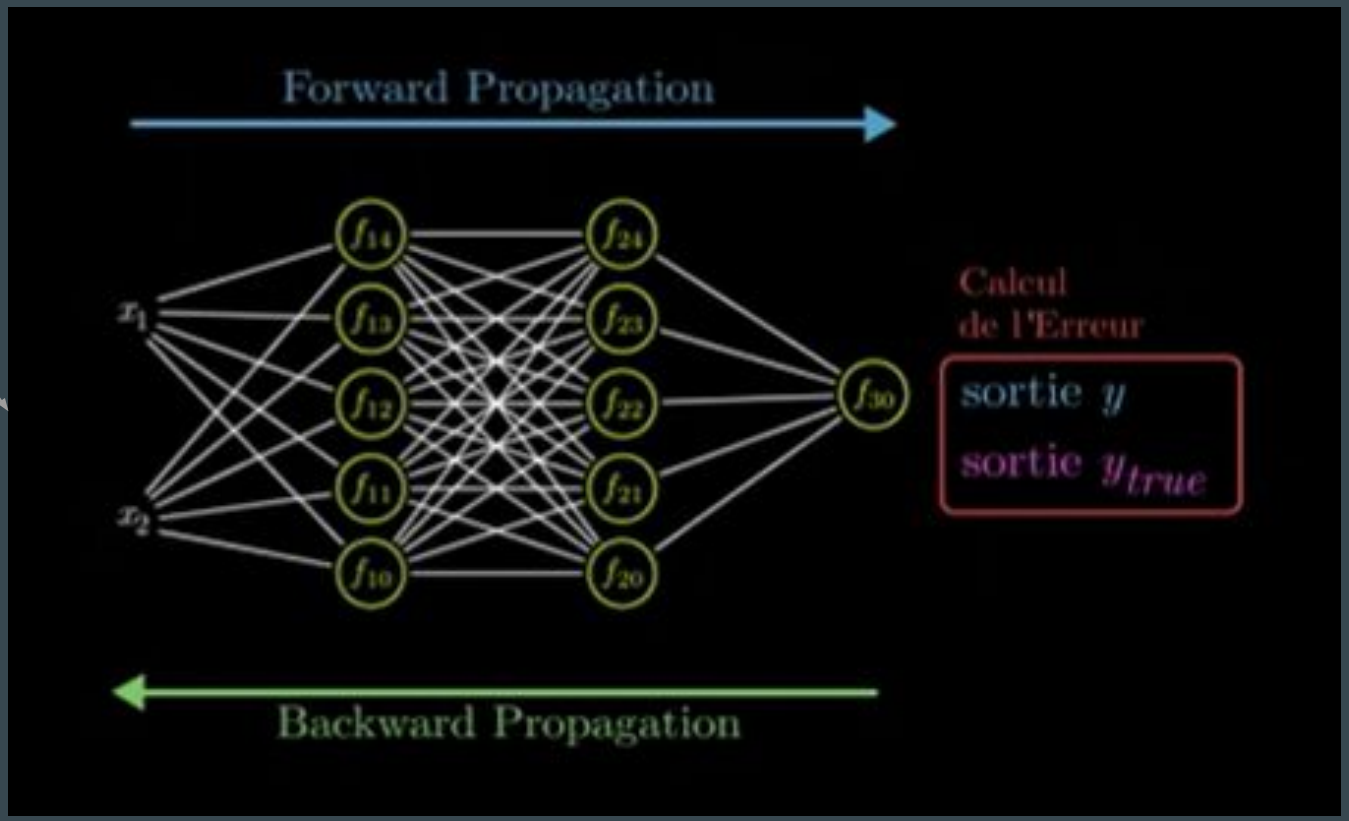
perceptron multicouche

Hyperbole donc différent du modèle linéaire



II Explication/Date

Schéma de modélisation et d'application toujours utilisé dans le monde du Deep Learning



III Application au projet

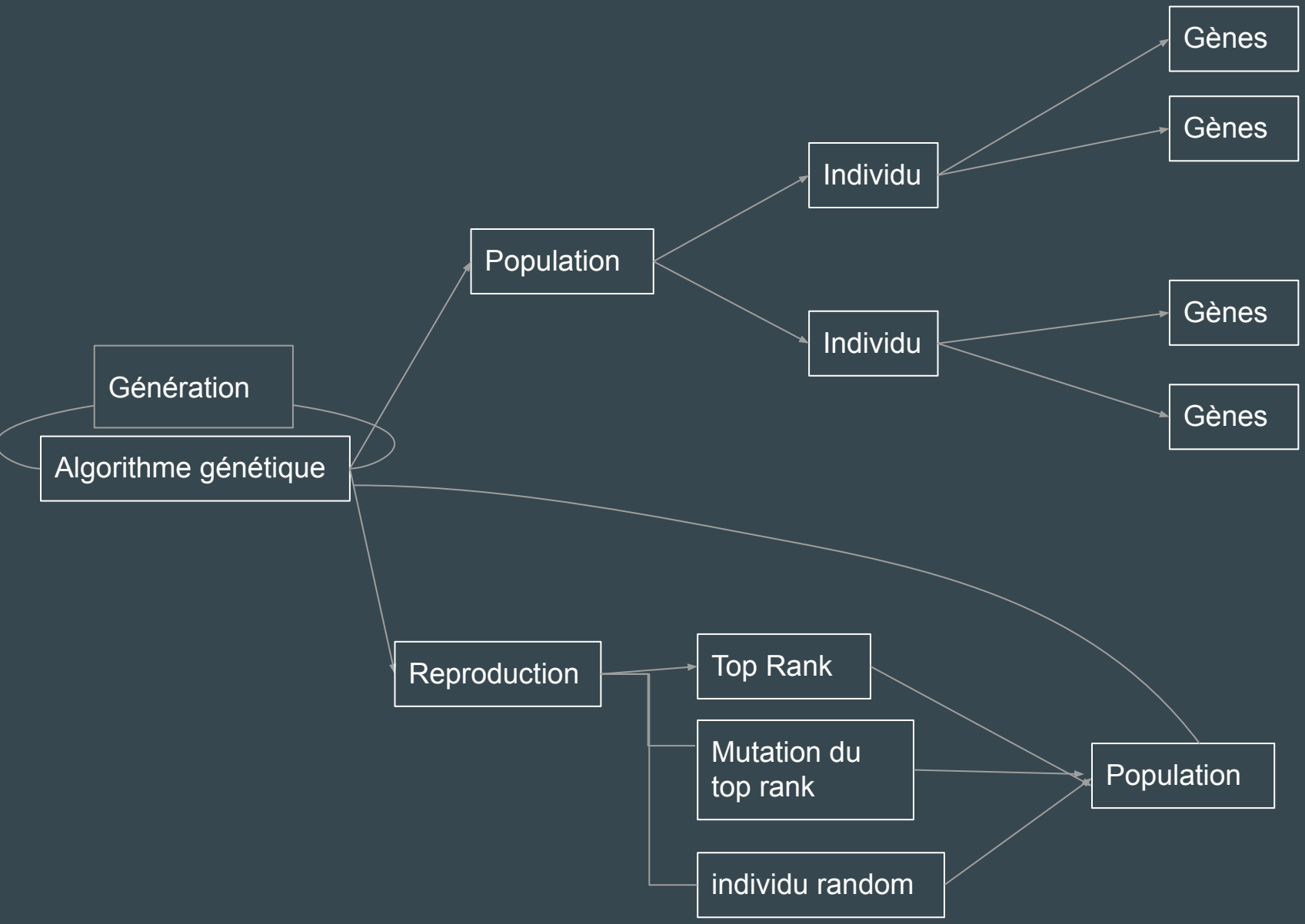
Projet Snake



- Faciliter de la formule de fitnesse
- peu de valeur à rentrer dans le réseaux
- peu de sous couche (2 à 3)

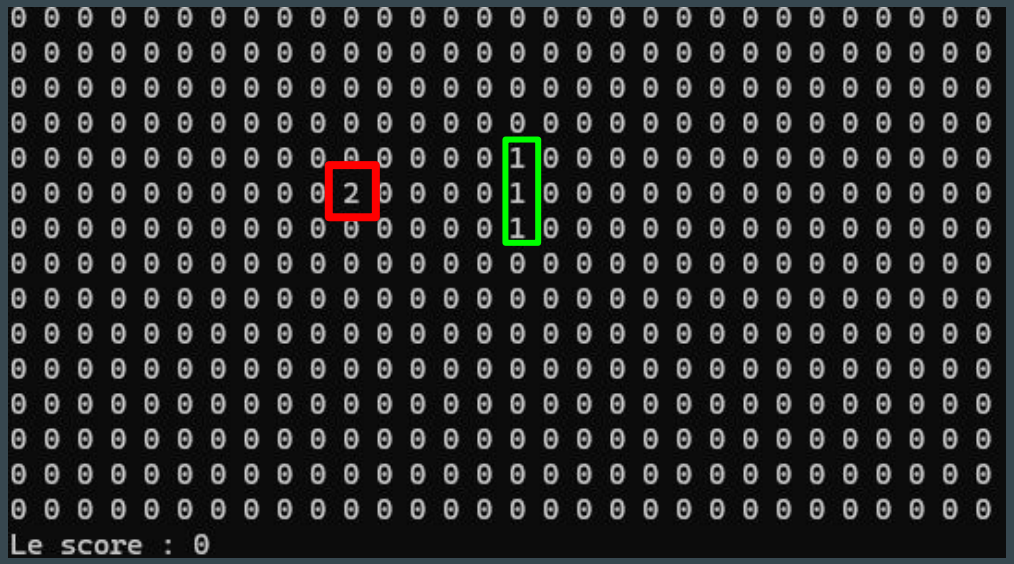
SnakeGame.c × main.py × GeneticAlgorithm.py

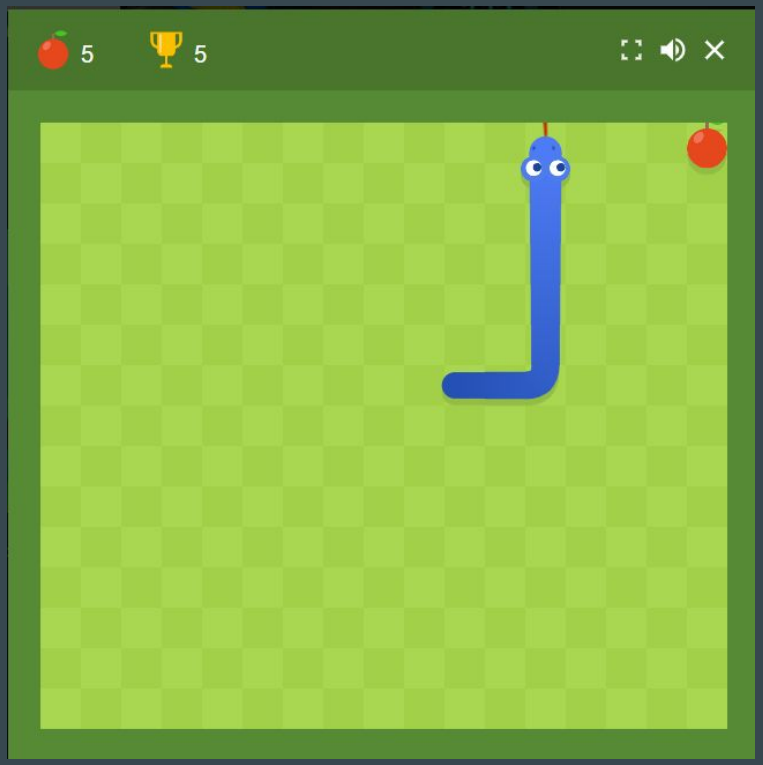
III Application au projet



III Application au projet

I) Le jeu





III Application au projet

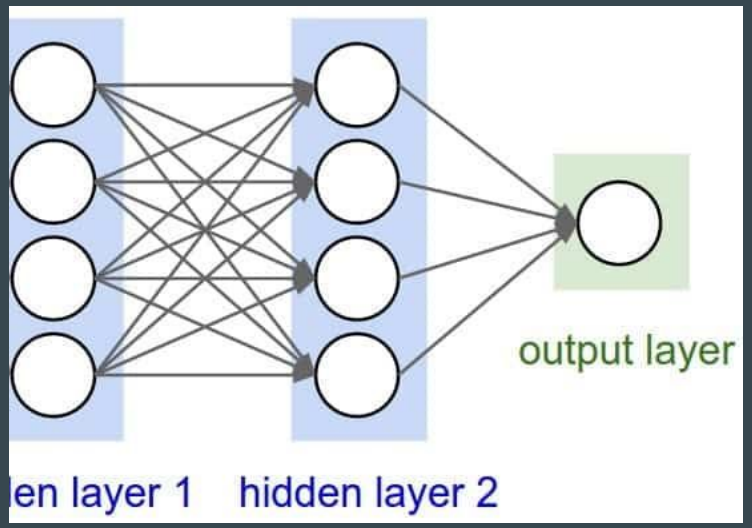
II) L'algo génétique

```
def generate_indiv(self, input_size: int, hidden_size: int, output_size: int):
```

$$\begin{bmatrix} -0.5 & 0.3 & \cdots & -0.8 \\ 0.6 & -0.1 & \cdots & 0.2 \\ \vdots & \vdots & \ddots & \vdots \\ 0.1 & -0.4 & \cdots & -0.3 \end{bmatrix}$$

$$\begin{bmatrix} 0.7 & -0.2 & \cdots & 0.5 \\ -0.3 & 0.4 & \cdots & -0.1 \\ \vdots & \vdots & \ddots & \vdots \\ -0.6 & 0.9 & \cdots & 0.3 \end{bmatrix}$$

$$\begin{bmatrix} 0.4 & -0.7 & \cdots & 0.6 \\ -0.5 & 0.8 & \cdots & -0.2 \\ \vdots & \vdots & \ddots & \vdots \\ 0.2 & -0.1 & \cdots & 0.3 \end{bmatrix}$$

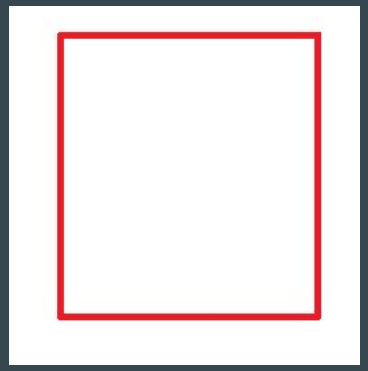


III Application au projet

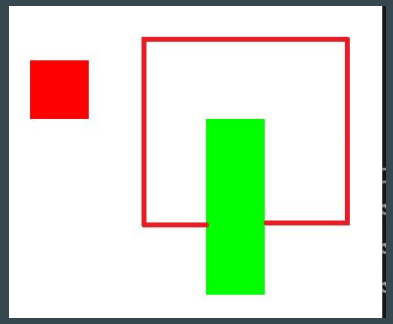
II) L'algo génétique

```
def vizualize_grid(self, grid: np.array, y_head: int, x_head: int):
```

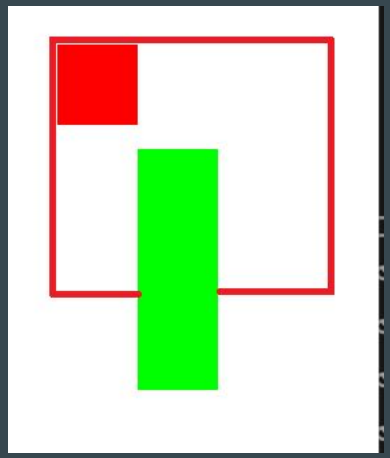
$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

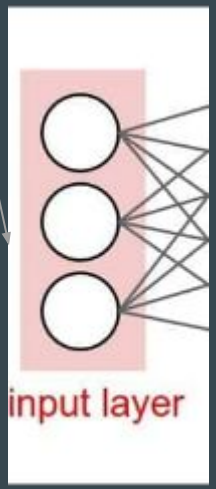


$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$



$$\begin{bmatrix} 2 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & -1 & 0 \end{bmatrix}$$



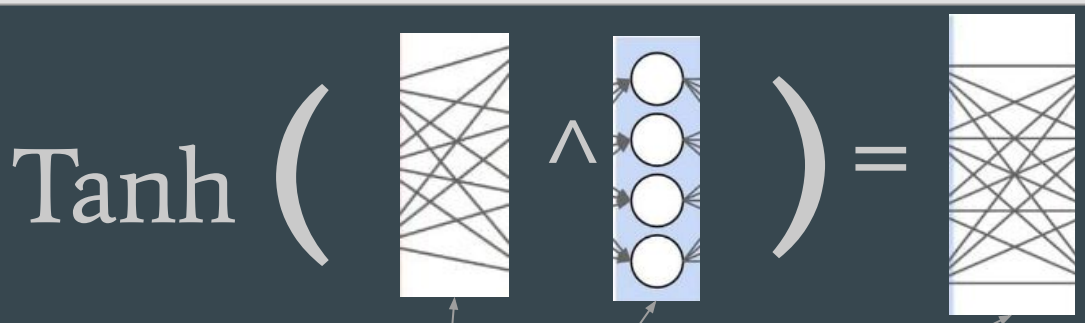


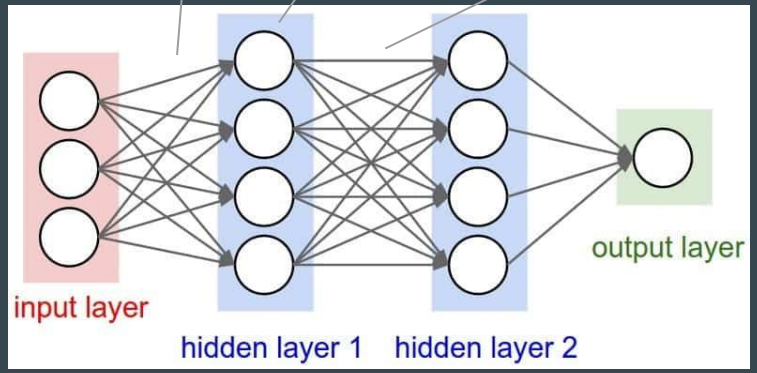
III Application au projet

II) L'algo génétique

```
def get_action(self, snake: List[Tuple[int, int]], grid: np.array):
```

Sortie du neurone = tanh(Produit scalaire des poids et des entrées)





III Application au projet

II) L'algo génétique

```
def reproduce(self, top_25: List[np.array]):
```

Population = ([],[],[],[],[],[],[],[],[],[],[],[],[],[],[],[],...[])

(...) = population
[...] = individu

Top_25= ([],[],[],[],[],[],[],[],[],[],[],[],[],[],[],[],...[])

Crossover

New_pop = ([],[],[],[],[],[],)

III Application au projet

II) L'algo génétique

```
for layer1, layer2 in zip(parent1, parent2):
    new_layer = np.copy(layer1)
    for i in range(new_layer.shape[0]):
        for j in range(new_layer.shape[1]):
            if random.uniform(0, 1) > 0.5:
                new_layer[i][j] = layer2[i][j]
    new_indiv.append(new_layer)
```

Layer 1

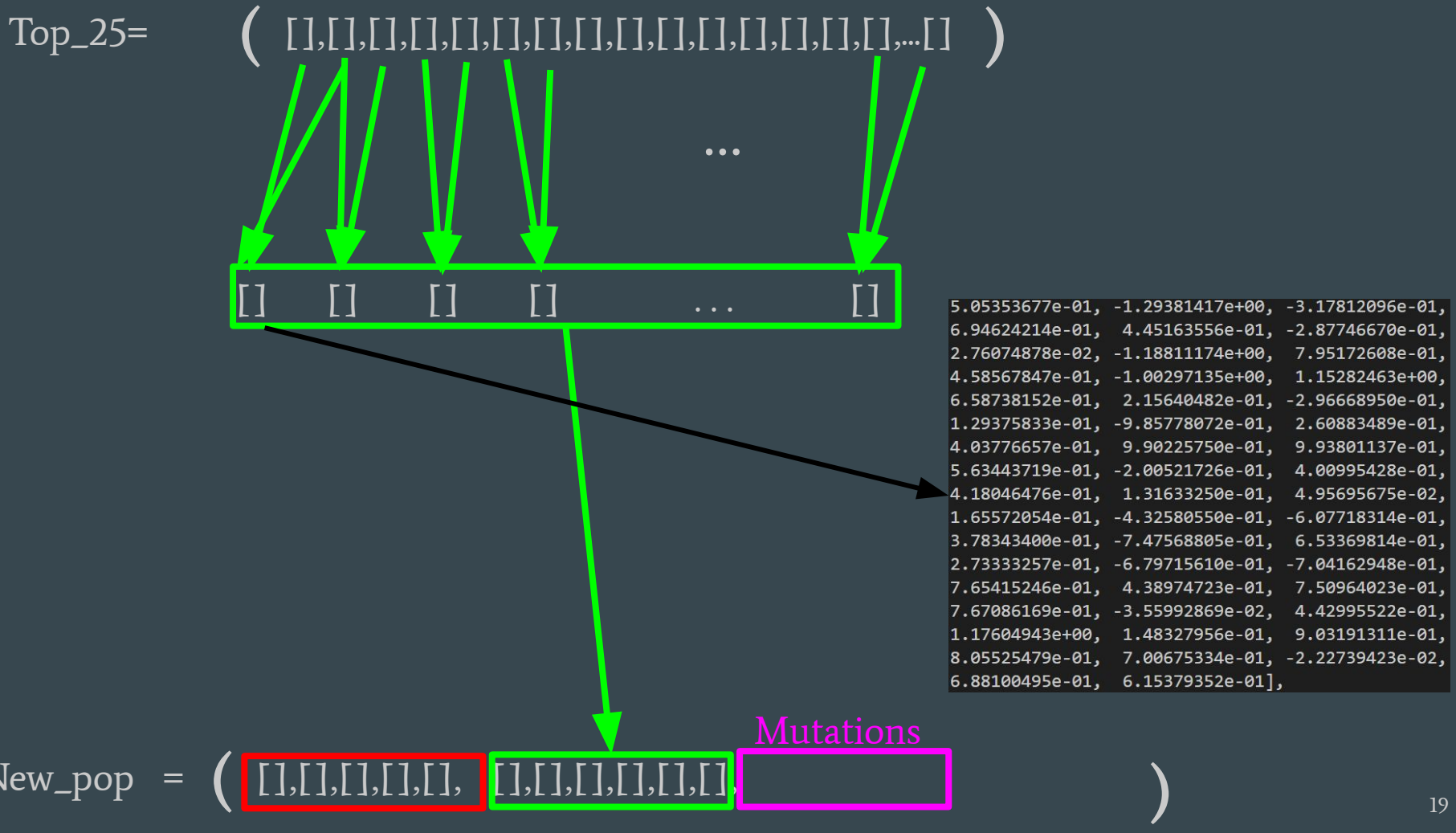
```
[-0.01896509, -0.97678295, -0.18030332, -0.39751238, -0.49540591,
 -0.09998459, -0.11015932,  0.56816157, -0.0358541 ,  0.42937681,
 -0.21240391, -0.44201767, -0.62150507, -0.25483112,  0.09490172,
 -0.57616423, -0.94833219,  0.11650879,  1.30334746,  0.17141837,
  0.61299316,  0.65796258, -1.00787312, -0.8071292 , -0.39419047,
 -0.39923509,  0.0689077 , -0.4650094 ,  0.29060809,  0.75408161,
  0.05561824,  0.12980769, -1.19978831,  0.64779361,  0.12486726,
 -0.5991151 , -0.03610767,  0.47057884,  0.6666744 ,  0.68944425,
  0.07141904,  1.09511771,  0.10380584,  0.3756704 ,  0.85945969,
 -0.97817826,  0.53897629, -0.35669292,  0.53646513,  0.64843978]
```

Layer 2

```
0.50535368, -1.29381417, -0.15558923, -0.69462421,  0.54002506,
0.28774667, -0.02760749, -1.19983343,  0.79313176, -0.63397901,
1.09298461,  1.09566759, -0.5566345 ,  0.21564048, -0.17899192,
0.12937583, -0.98577807,  0.26088349,  0.43499087,  0.77294627,
1.0386711 ,  0.33373778, -0.20052173,  0.40099543, -0.46765499,
0.00266434,  0.27933723, -0.02247921, -0.70072708, -0.60771831,
0.3783434 , -0.7475688 ,  0.65336981, -0.27333326, -0.74136151,
0.70416295, -0.76541525,  0.43897472,  0.75096402, -1.02585994,
0.05979564,  0.44299552, -0.99133682,  0.22501653,  0.90319131,
0.87189323,  0.75654221,  0.04210268,  0.6881005 ,  0.61537935]
```

III Application au projet

II) L'algo génétique

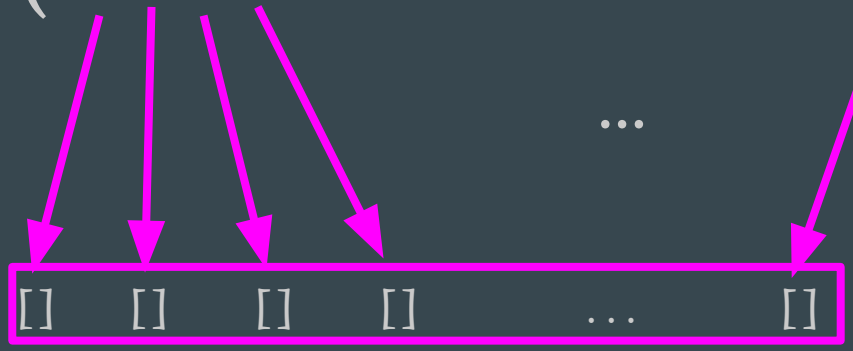


III Application au projet

II) L'algo génétique

```
def mutate(self, indiv: np.array):
```

Top_25= ([],[],[],[],[],[],[],[],[],[],[],[],[],[],[],[],...[])




```
mutation_chance = 0.5,
mutation_size = 0.2,
```

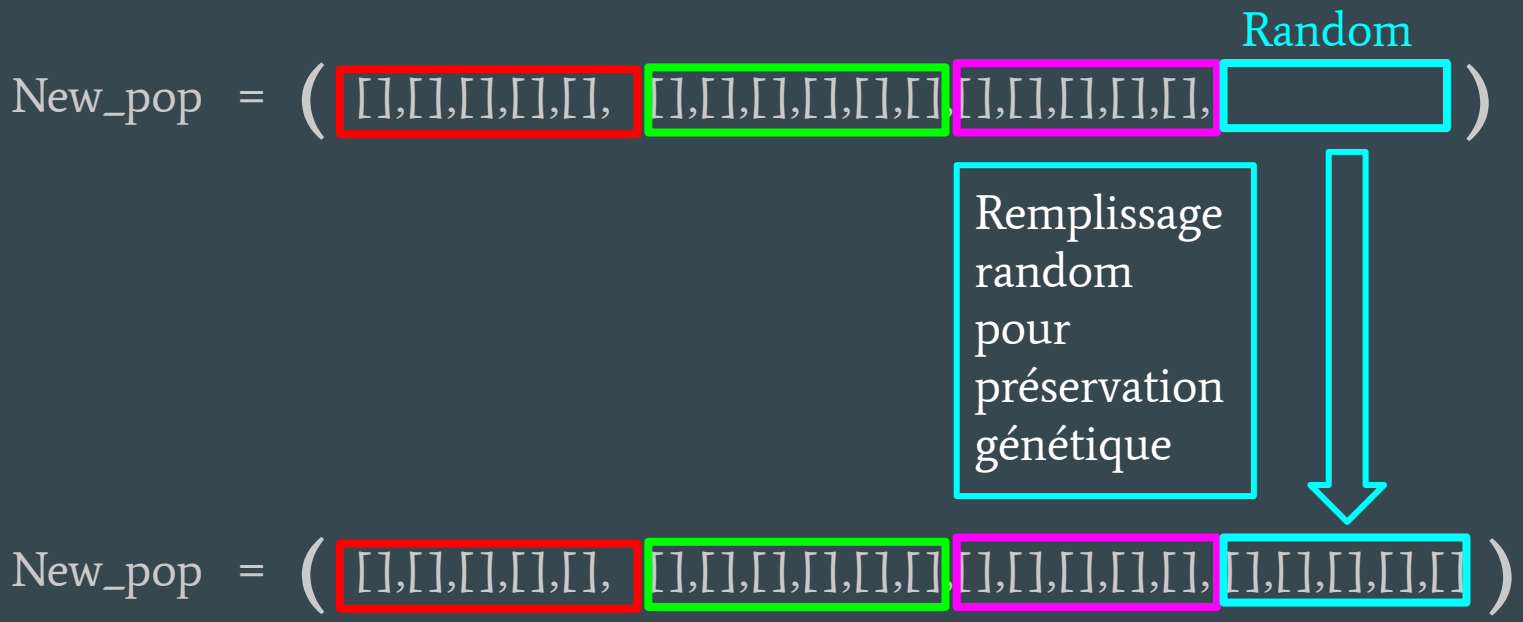
```
0.50535368, -1.29381417, -0.15558923, -0.69462421,  0.54002506,
0.28774667, -0.02760749, -1.19983343,  0.79313176, -0.63397901,
1.09298461,  1.09566759, -0.5566345 ,  0.21564048, -0.17899192,
0.12937583, -0.98577807,  0.26088349,  0.43499087,  0.77294627,
1.0386711 ,  0.33373778, -0.20052173,  0.40099543, -0.46765499,
0.00266434,  0.27933723, -0.02247921, -0.70072708, -0.60771831,
0.3783434 , -0.7475688 ,  0.65336981, -0.27333326, -0.74136151,
0.70416295, -0.76541525,  0.43897472,  0.75096402, -1.02585994,
0.05979564,  0.44299552, -0.99133682,  0.22501653,  0.90319131,
0.87189323,  0.75654221,  0.04210268,  0.6881005 ,  0.61537935]
```

0.50535368 + 0.1*0.2 =0.8535368

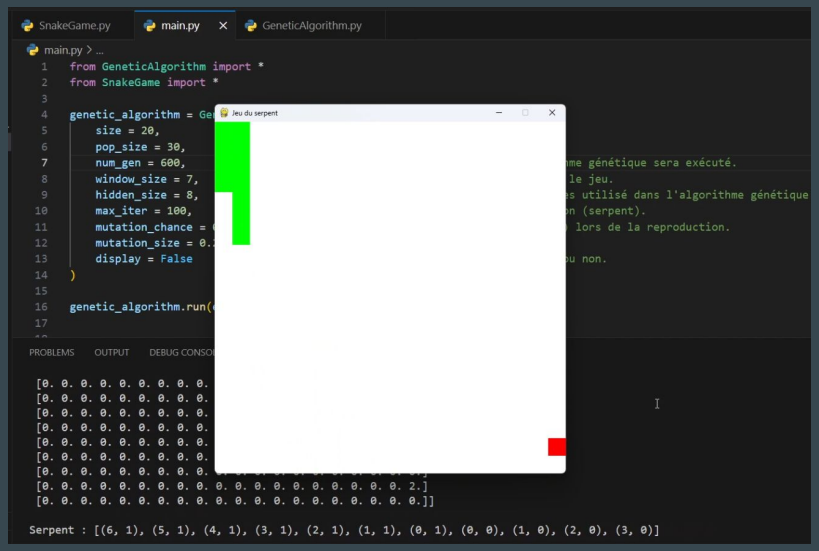
```
for i in range(new_layer.shape[0]):
    for j in range(new_layer.shape[1]):
        rand = random.uniform(0, 1)
        if rand <= self.mutation_chance:
            new_layer[i][j] += random.uniform(-1, 1) * self.mutation_size
new_indiv.append(new_layer)
```

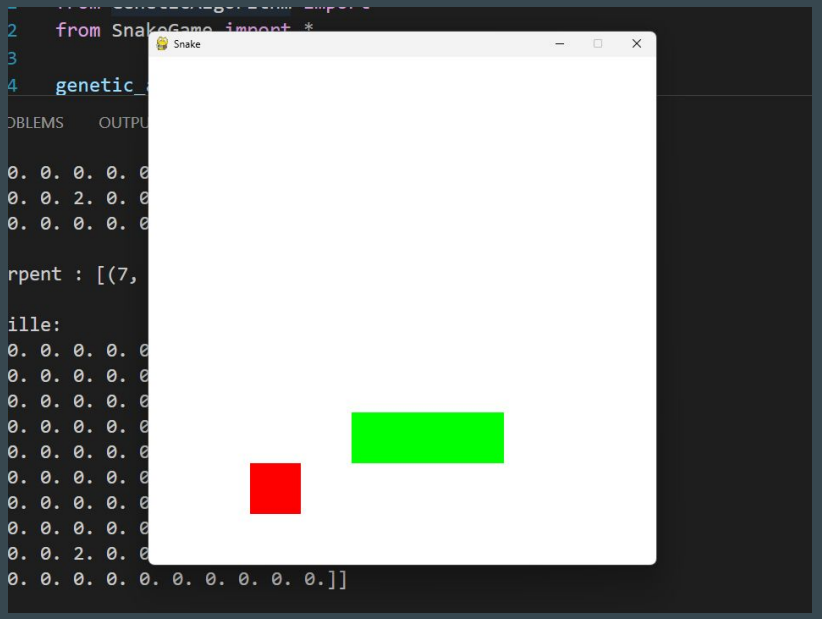
III Application au projet

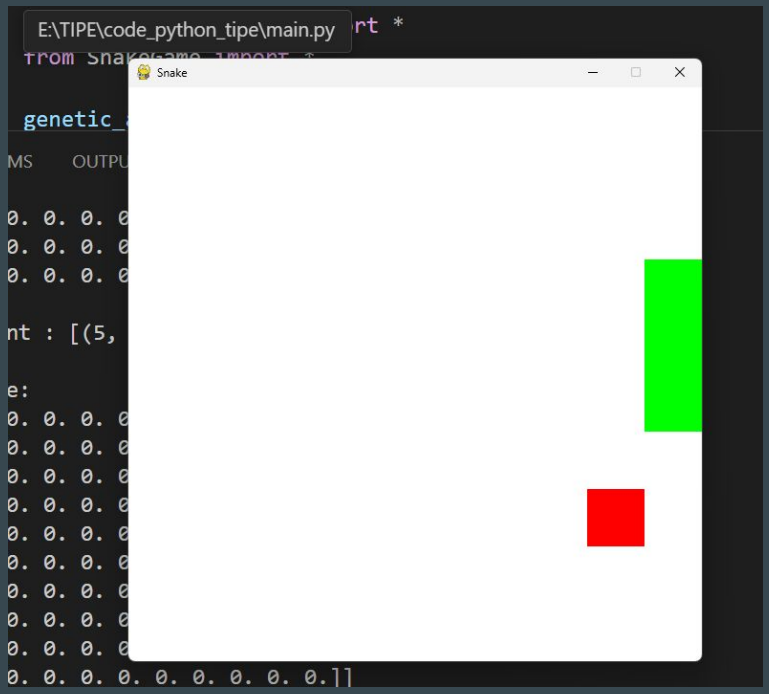
II) L'algo génétique

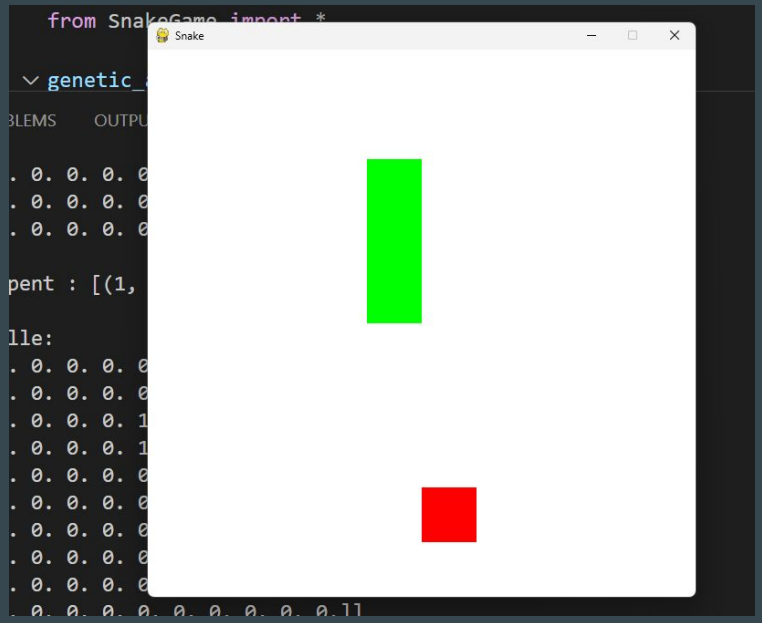


III Application au projet

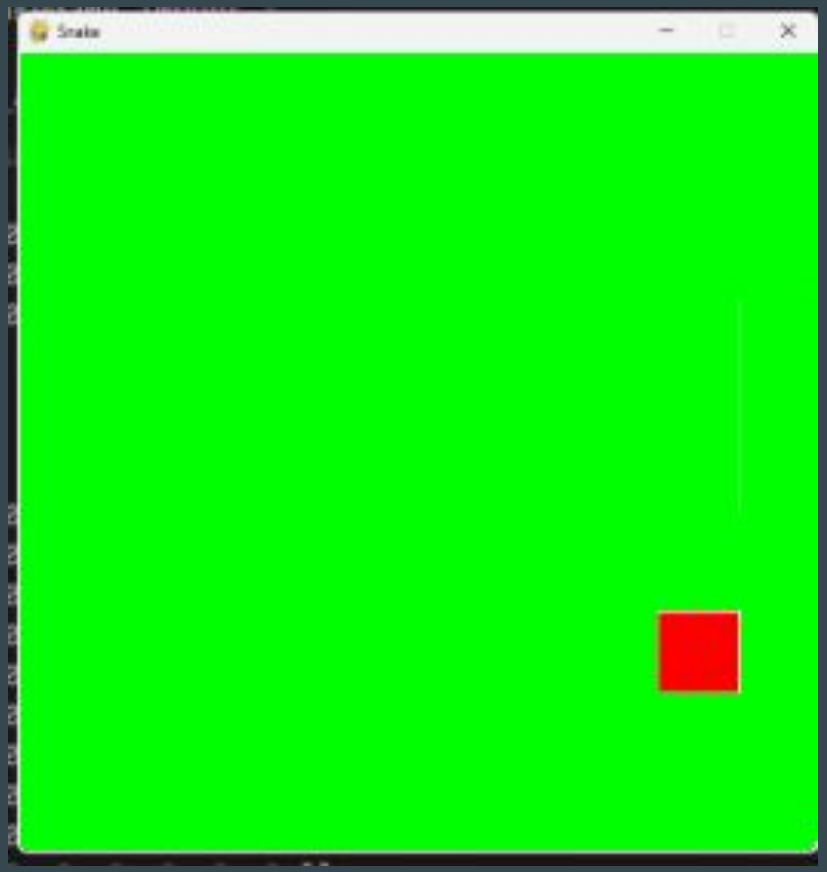




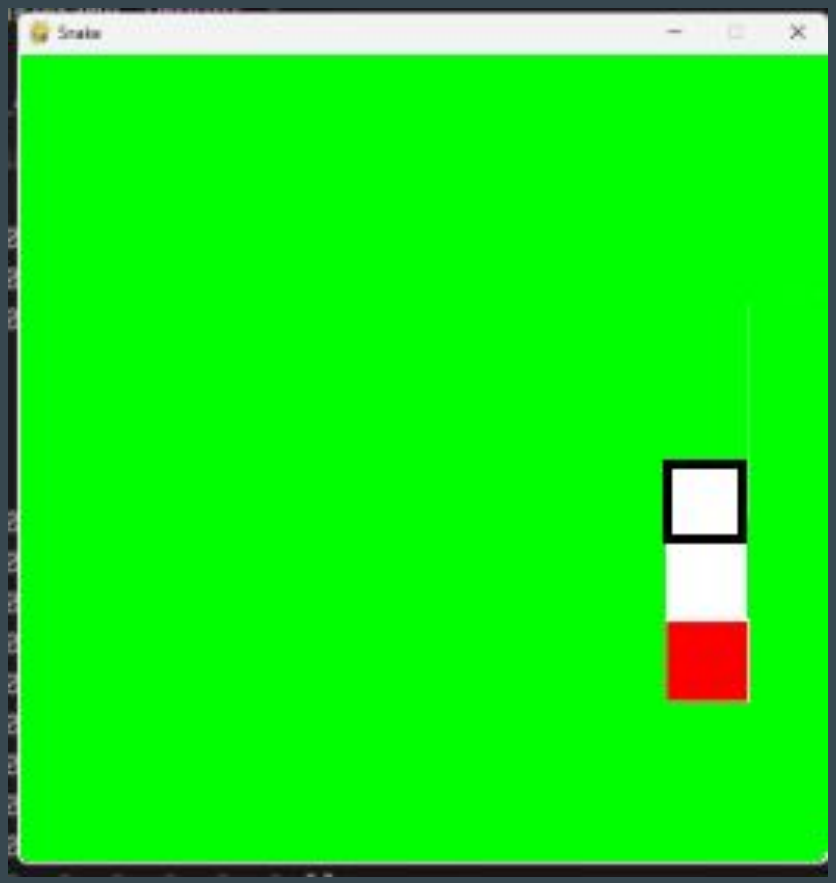




III Application au projet



Génération : 566
individu :18
fitness : 26854



Génération : 564
individu : 8
fitness : 25482

Merci pour votre attention

IV Index + Annexe

-https://www.youtube.com/watch?v=F63GNXGHVwM
-https://www.youtube.com/@SethBling
-https://www.youtube.com/@MachineLearnia
-https://www.google.com/search?q=image+machine+learnig&rlz=1C1RXMK_frFR1057FR1057&oq=image+machine+learnig&gs_lcrp=EgZjaHJvbWUyBggAEEUYOTIJCAEQABgNGIAEMgkIAhAAGA0YgAQyCAgDEAAYFhgeMggIBBAAGBYYHjIICAUQABgWGB4yCAgGEAAYFhgeMggIBxAAGBYYHjIKCAgQABgPGBYYHjIKCAkQABgPGBYYHtIBCDYwODlqMGo3qAIAsAIA&sourceid=chrome&ie=UTF-8#imgrc=8OukywakkwfAgM
-https://www.google.com/search?rlz=1C1RXMK_frFR1057FR1057&sxsrf=APwXEdcT4HbMgLy8Cvxqj4oAp2pVcqliGA:1687346984597&q=mario+bros+usa+japan&tbm=isch&sa=X&ved=2ahUKEwjnvYStodT_AhVaXaQEHYlyCIwQ0pQJegQICRAB&biw=958&bih=951&dpr=1
-https://fr.wikipedia.org/wiki/Apprentissage_automatique
-https://en.wikipedia.org/wiki/Machine_learning
-https://fr.wikipedia.org/wiki/Apprentissage_profond
-https://www.youtube.com/watch?v=u5xCl1bSe6o&t=0s
-https://www.youtube.com/watch?v=ncj_hBfRt-Y&t=0s
-https://www.youtube.com/watch?v=Wo5dMEP_BbI&t=0s

IV Index + Annexe

-https://www.google.com/search?q=the+end+images&rlz=1C1RXMK_frFR1057FR1057&oq=the+end+image&gs_lcrp=EgZjaHJvbWUqBwgAEAAYgAQyBwgAEAAYgAQyBggBEEUYOTIHCAIQABiABDIHCAMQABiABDIHCAQQABiABDIHCAUQABiABDIHCAYQABiABDIHCAcQABiABDIHCAgQABiABDIHCAkQABiABNIBCDIzMTVqMWo3qAIAsAIA&sourceid=chrome&ie=UTF-8#imgrc=fVxDQO3PfNmuCM
https://www.google.com/search?rlz=1C1RXMK_frFR1057FR1057&sxsrf=APwXEdfBqs2sURC3TyU66FCmjFh39iQt-A:1687449185933&q=zombie+images&tbm=isch&sa=X&ved=2ahUKEwjG1baKntf_AhVGU6QEHdyOBtYQ0pQJegQIChAB&biw=1053&bih=951&dpr=1#imgrc=uJjHt6vG6EI9qM

IV Index + Annexe

I) Le jeux

```c
void pomme(int matrix[COLS][ROWS], int *x, int *y)
    *x = rand() % COLS;
    *y = rand() % ROWS;
    if (matrix[*x][*y] == 0)
        matrix[*x][*y] = 2;
}
```

```c
for (int i = 0; i < COLS; i++){
    for (int j = 0; j < ROWS; j++){
        matrix[i][j] = 0;
    }
}
```

IV Index + Annexe

I) Le jeux

```
int snake[100][2];
int snake_len = 3;
for(int i = 0; i < snake_len; i++){
    snake[i][0] = COLS / 2;
    snake[i][1] = (ROWS / 2) + i;
}
```

```
for(int i = snake_len; i < 100; i++){
    snake[i][0] = -1;
    snake[i][1] = -1;
}
```

IV Index + Annexe

I) Le jeux

```
while(!quit){
    int ch = getch();
    int new_direction = direction;
    switch(ch){
    case KEY_UP:
        if (direction != 2)
            new_direction = 0;
        break;
    case KEY_RIGHT:
        if (direction != 3)
            new_direction = 1;
        break;
    case KEY_DOWN:
        if (direction != 0)
            new_direction = 2;
        break;
```

```
case KEY_LEFT:
    if (direction != 1)
        new_direction = 3;
    break;
case 'e':
    quit = 1;
}

if (abs(new_direction - direction) != 2)
    direction = new_direction;
```

```
if (direction == 0) snake[0][1]--;
if (direction == 1) snake[0][0]++;
if (direction == 2) snake[0][1]++;
if (direction == 3) snake[0][0]--;
```

IV Index + Annexe

I) Le jeux

```
if (snake[0][0] == appleX && snake[0][1] == appleY){
    snake_len++;
    score++;
    pomme(matrix, &appleX, &appleY);
    printf("Pomme mangée!\n");
}

if (snake[0][0] < 0 || snake[0][0] >= COLS ||
    snake[0][1] < 0 || snake[0][1] >= ROWS){
        quit = 1;
}
```

```
matrix[appleX][appleY] = 2;
clear();
affichage(matrix, score);
refresh();
```

VI Application au projet

II) L'algo génétique

```python
import math
import random
import numpy as np
from typing import List, Tuple
from SnakeGame import SnakeGame

LEFT = (0, -1)
RIGHT = (0, 1)
UP = (-1, 0)
DOWN = (1, 0)

MOVES = [LEFT, RIGHT, UP, DOWN]
```

```python
class GeneticAlgorithm(object):
    def __init__(
        self,
        size: int,
        pop_size: int,
        num_gen: int,
        window_size: int,
        hidden_size: int,
        max_iter=100,
        mutation_chance=0.1,
        mutation_size=0.1,
        display=False,
```

IV Index + Annexe

II) L'algo génétique

```python
def get_fitness(self, game: SnakeGame):
    score = len(game.snake) - game.snake_size
    return game.iter * 2 ** score
```

```python
def one_generation(self, eating=True):
    scores = [0 for _ in range(self.pop_size)]
    max_score = 0
    for i in range(self.pop_size):
        if self.display:
            print(f"Snake {i} playing...")
        self.current_individual = self.pop[i]
        game = SnakeGame(
            self.size, self.size // 3, self, eating=eating, max_iter=self.max_iter
        )
        outcome = game.run()
        if eating:
            score = self.get_fitness(game)
        else:
            score = game.iter
        scores[i] += score
        if self.display:
            print(f"Score: {score}\n")
            if outcome == 0:
                print(f"Snake {i} survived !\n")
        if score > max_score:
            max_score = score
            if self.display:
                print(f"Max score: {max_score}, at ID {i}\n")
    top_25_indexes = list(np.argsort(scores))[3 * (self.pop_size // 4) : self.pop_size]
    print(f"Scores : {scores}")
    top_25 = [self.pop[i] for i in top_25_indexes][::-1]
    self.pop = self.reproduce(top_25)
```

IV Index + Annexe

III) Main

```python
from GeneticAlgorithm import *
from SnakeGame import *

genetic_algorithm = GeneticAlgorithm(
    size = 20,              # Taille de la population de gènes.
    pop_size = 30,          # Taille de la population de solutions.
    num_gen = 3000,          # Nombre de générations sur lesquelles l'algorithme génétique sera exécuté.
    window_size = 7,        # Taille de la fenêtre de vision du serpent dans le jeu.
    hidden_size = 8,        # Taille de la couche cachée du réseau de neurones utilisé dans l'algorithme génétique.
    max_iter = 100,         # Nombre maximal d'itérations pour chaque solution (serpent).
    mutation_chance = 0.5, # Probabilité de mutation d'un gène (une solution) lors de la reproduction.
    mutation_size = 0.2,   # Taille de la mutation lorsqu'elle se produit.
    display = False
)

genetic_algorithm.run(eating=True)
```

IV Index + Annexe

$$L = \prod_{i=1}^{m} P(Y = y_i)$$

Produit de la probabilité des plausibilités vis à vis de vrai données

Bernoulli

$$L = \prod_{i=1}^{m} a_i^{y_i} \times (1 - a_i)^{1-y_i}$$

Vraisemblance

IV Index + Annexe

On part de la fonction de vraisemblance :

$$L = \prod_{i=1}^{m} a_i^{y_i} \times (1 - a_i)^{1-y_i}$$

$$log(L) = log(\prod_{i=1}^{m} a_i^{y_i} \times (1 - a_i)^{1-y_i})$$

$$= \sum_{i=1}^{m} log(a_i^{y_i} \times (1 - a_i)^{1-y_i})$$

Rappel :
$log(ab) = log(a) + log(b)$

$$= \sum_{i=1}^{m} log(a_i^{y_i}) + log((1 - a_i)^{1-y_i})$$

IV Index + Annexe

Ainsi on trouve :

$$= \sum_{i=1}^{m} y_i \, log(a_i) + (1 - y_i) \, log(\ 1 - a_i\ )$$

Rappel :
$log(a^y) = ylog(a)$

$$L = -\frac{1}{m} \sum_{i=1}^{m} y_i log(a_i) + (1 - y_i) log(1 - a_i)$$

III Fonctions Mathématique

$$a(z) = \frac{1}{1 + e^{-z}}$$

Fonction Sigmoïde

$$P(Y = y) = a(z)^y \times (1 - a(z))^{1-y}$$

Loi de Bernoulli

$$L = -\frac{1}{m} \sum_{i=1}^{m} [y_i \log(a_i) + (1 - y_i) \log(1 - a_i)]$$

Loss Function

$$L = \prod_{i=1}^{m} a_i^{y_i} \times (1 - a_i)^{1-y_i}$$

Fonction de vraisemblance

III Fonctions Mathématique

On va pouvoir se servir de la fonction LogLoss pour minimiser les erreurs grâce à la Descente de gradient (dérivations du LogLoss)

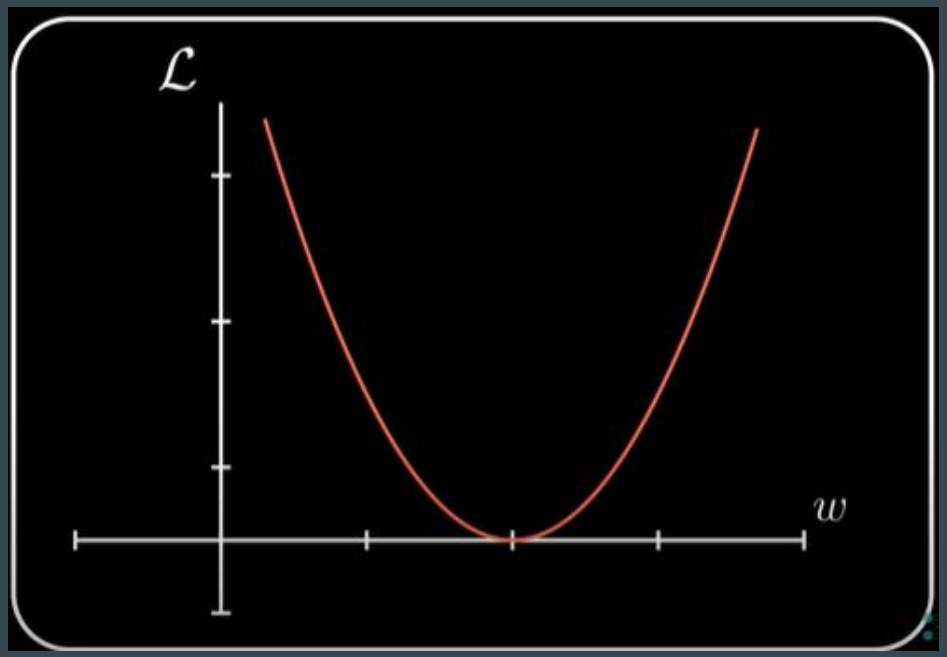
$$W_{t+1} = W_t - \alpha \frac{\partial \mathcal{L}}{\partial W_t}$$

W_{t+1} : Paramètre W à l'instant t+1

W_t : Paramètre W à l'instant t

α : Pas d'apprentissage positif

$\frac{\partial \mathcal{L}}{\partial W_t}$: Gradient à l'instant t



III Fonctions Mathématique

$$f = w_1 x_1 + w_2 x_2 + b$$

$$Z = X \cdot W + b \qquad X = \begin{bmatrix} x_1^{(1)} & \cdots & x_n^{(1)} \\ \vdots & \ddots & \vdots \\ x_1^{(m)} & \cdots & x_n^{(m)} \end{bmatrix} \in \mathbb{R}^{m \times n} \qquad W = \begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix} \in \mathbb{R}^{n \times 1}$$

$$\mathcal{L} = -\frac{1}{m} \sum_{i=1}^{m} y^{(i)} \times log(a^{(i)}) + (1 - y^{(i)}) \times log(1 - a^{(i)})$$

$$\mathcal{L} = -\frac{1}{m} \sum y \times log(A) + (1 - y) \times log(1 - A)$$

$$a(z) = \frac{1}{1 + e^{-z}}$$

$$A = \frac{1}{1 + e^{-z}}$$